NFL Modeling - SVM

Chandler Grote

2024-12-06

Modeling

In this document we will develop a statistical model to predict whether the offense will execute a run or a pass play. The model will include the following variables

-game_id: Unique identifier for the game

-play_id: Unique identifier for a specific play

-possession_team: Identifies the offensive team

-week: Identifies which week of the season the game was in.

-run_pass: Indicates if the play was a run of a pass (response variable)

-down: Identifies which down the offensive team was on for a particular play.

-yards_to_go: The number of yards till a first down.

-yards_to_endzone: The number of yeards till the opponents endzone.

-redzone: Indicates whether the offense is within 20 yards of the opponents endzone.

-sec_in_half: The number of seconds remaining on the play clock in the current half.

-offense_formation: Categorical variable indicating what formation the offense uses.

-receiver_alingment: Enumerated as 0x0, 1x0, 1x1, 2x0, 2x1, 2x2, 3x0, 3x1, 3x2 (text)

-epa_lag: Serial correllation of expected points added.

-curr_win_percentage: Win percentage for the current offensive team.

-form_change: Uses shifts to determine if the offense changed formation before the ball was snapped.

-wr_motion: Indicates if a wide receiver moved after the line was set.

-te_motion: Indicates if a tight end moved after the line was set.

-rb_motion: Indicates if a running back moved after the line was set.

-wr_atsnap: Indicates if a wide receiver was in motion at the time of the snap.

-te_atsnap: Indicates if a tight end was in motion at the time of the snap.

-rb_atsnap: Indicates if a running back was in motion at the time of the snap.

Modeling

Cleaning the data frame

```
nfl_df <- read.csv("data/modeling_data_frame.csv")

# define unwanted columns to make modeling formula easier
unwanted_columns <- c("game_id", "play_id")

# clean the df to remove unwanted columns and any rows containing NA values
df_clean <- nfl_df %>% select(-all_of(unwanted_columns)) %>% drop_na()

model_data <- nfl_df %>%
  mutate(run_pass = as.factor(run_pass),
         down = as.factor(down),
         offense_formation = as.factor(offense_formation),
         form_change = as.factor(form_change),
  ) %>%
  na.omit() %>% select(-all_of(unwanted_columns))

#Previously removed receiver alignment to ensure models run
#Now remove formations not commonly used by teams

# Create a data frame of all combinations to remove
remove_combinations <- data.frame(
  possession_team = c(
    "ARI", "ARI", "ATL", "BAL", "BUF", "BUF", "BUF", "CAR", "CAR", "CAR",
    "CHI", "CLE", "CLE", "DAL", "DEN", "DEN", "DET", "DET", "HOU", "IND",
    "IND", "IND", "JAX", "JAX", "KC", "KC", "LA", "LV", "LV", "MIA",
    "MIN", "NE", "NE", "NO", "NYG", "NYJ", "NYJ", "NYJ", "NYJ", "PIT",
    "PIT", "SEA", "SEA", "SF", "TEN", "TEN", "WAS", "WAS"
  ),
  offense_formation = c(
    "I_FORM", "WILDCAT", "JUMBO", "WILDCAT", "JUMBO", "PISTOL", "WILDCAT", "JUMBO", "PISTOL", "WILDCAT"
    "WILDCAT", "PISTOL", "WILDCAT", "WILDCAT", "JUMBO", "WILDCAT", "JUMBO", "PISTOL", "PISTOL", "JUMBO"
    "PISTOL", "WILDCAT", "JUMBO", "WILDCAT", "JUMBO", "WILDCAT", "PISTOL", "JUMBO", "WILDCAT", "WILDCAT
    "PISTOL", "PISTOL", "WILDCAT", "JUMBO", "JUMBO", "I_FORM", "JUMBO", "PISTOL", "WILDCAT", "JUMBO",
    "WILDCAT", "JUMBO", "WILDCAT", "JUMBO", "JUMBO", "WILDCAT", "JUMBO", "WILDCAT"
  )
)

# Filter the dataset using anti_join
model_data <- model_data %>%
  anti_join(remove_combinations, by = c("possession_team", "offense_formation"))
```

Define getting the bye week function

```
get_bye_week <- function(weeks){
  # this gets the week number that is missing from the given weeks
  bye_week <- seq(9)[!(seq(9) %in% weeks)]

  # return NA if all weeks of data are present
```

```
  if(identical(bye_week, integer(0))){
    return(NA)
  }else{
    return(bye_week)
  }
}

test_data_for_augment <- function(model, data){

}
```

SVM modeling

```
## --NOTE: Eval has been set to FALSE for this chunk.
## This chunk takes approximately 2 hours to finish on my machine
## The data structure output from this chunk has been saved seperately if desired

# set seed for reproduction
set.seed(445)
# initialize model info storing method
svm_model_info <- list("nfl_team" = unique(df_clean$possession_team),
                       "bye_week" = as.list(seq(32)),
                       "train_fit" = as.list(seq(32)),
                       "test_fit" = as.list(seq(32)),
                       "conf_mat" = as.list(seq(32)))


# define svm specification for model
svm_linear_spec <- svm_poly(degree = 1) %>%
  set_mode("classification") %>%
  set_engine("kernlab", scaled = FALSE)

# define svm tuning spec
svm_linear_tune_spec <- svm_poly(degree = 1, cost = tune("cost")) %>%
  set_mode("classification") %>%
  set_engine("kernlab", scaled = FALSE)


for(i in c(1:32)){
  # get team abreviation for loop
  team <- svm_model_info$nfl_team[i]
  # print statemtn for loop checking during iterations
  print(paste("Working on team", team))
  # select only the current team
  team_df <- model_data %>% dplyr::filter(possession_team == team)
  # find which weeks were played for this team
  weeks <- sort(unique(team_df$week))
  # get the bye week for the team
  svm_model_info$bye_week[[i]] <- get_bye_week(weeks)
  # set training data to everything except last 3 weeks
  team_train <- team_df %>%
    filter(week <= weeks[(length(weeks)-3)]) %>% select(-possession_team, -week)
```

```r
  # set testing data to the last 3 weeks of data
  team_test <- team_df %>%
    filter(week > weeks[(length(weeks)-3)]) %>% select(-possession_team, -week)
  # these lines make sure these categorical variables only share info so augment doesnt error out
  team_train <- team_train %>% filter(offense_formation %in% team_test$offense_formation,
                                      receiver_alignment %in% team_test$receiver_alignment)

  team_test <- team_test %>% filter(offense_formation %in% team_train$offense_formation,
                                    receiver_alignment %in% team_train$receiver_alignment)

  # set train cv and cost tuning df
  team_train_10foldcv <- vfold_cv(team_train, v = 5)
  df_cost <- grid_regular(cost(), levels = 10)

  # recipe for tuning
  svm_linear_rec <- recipe(run_pass ~ ., data = team_train)
  # workflow for tuning
  svm_linear_wf <- workflow() %>%
    add_model(svm_linear_tune_spec) %>%
    add_recipe(svm_linear_rec)
  # tune fit
  tune_fit <- svm_linear_wf %>%
    tune_grid(resamples = team_train_10foldcv, grid = df_cost)
  # extract the best cv fit
  svm_linear_final <- finalize_workflow(svm_linear_wf, select_best(tune_fit, metric = "accuracy"))

  # fit svm model to best tuned version
  svm_train_fit <- svm_linear_final %>% fit(data = team_train)
  # store model
  svm_model_info$model[[i]] <- svm_train_fit
  # predict with test observations
  test_predict <- svm_train_fit %>% augment(new_data = team_test)
  # store test prediction
  svm_model_info$test_fit[[i]] <- test_predict
  # run test prediction through confusion matrix
  test_conf_mat <- confusionMatrix(test_predict$.pred_class, test_predict$run_pass)
  # store confusion matrix for each team
  svm_model_info$conf_mat[[i]] <- test_conf_mat
}
```

save the data

```r
saveRDS(svm_model_info, "SVM Model Info by Team.rds")
```

Graphing

```r
svm_model_info <- readRDS("SVM Model Info by Team.rds")
```

```r
# list team abbreviations based on conference
afc <- c("KC", "BUF", "PIT", "LAC", "HOU", "BAL", "DEN", "IND", "MIA", "CIN",
         "CLE", "NYJ", "TEN", "NE", "JAX", "LV")
```

```r
nfc <- c("DET", "PHI", "MIN", "GB", "WAS", "SEA", "ATL", "TB", "ARI", "LAR",
         "SF", "DAL", "NO", "CHI", "CAR", "NYG")
```

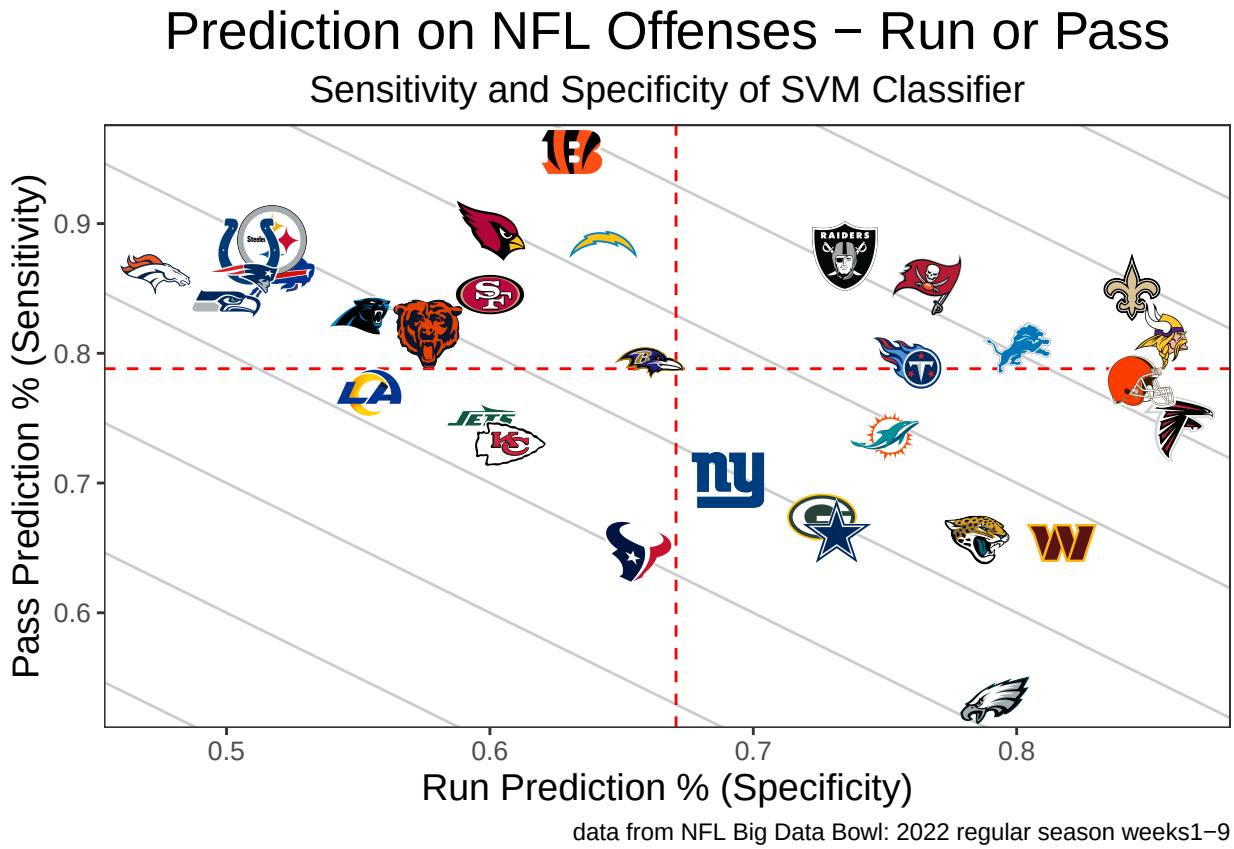
```r
nfl_data <- read.csv("data/modeling_data_frame.csv")

run_pass_prop <- nfl_data %>%
  group_by(possession_team) %>%
  summarise(tot_run = sum(run_pass == "run"),
            tot_pass = sum(run_pass == "pass")) %>%
  ungroup() %>%
  rename(team = "possession_team")

win_loss <- read.csv("data/win_loss.csv")
```

```r
# initialize df
svm_data <- data.frame(team = unique(svm_model_info$nfl_team),
                       pass_1 = -1, pass_0 = -1, run_1 = -1, run_0 = -1,
                       mn_pvalue = 1, acc_pvalue = 1,
                       bye_week = 0, train_error = 0, num_vec = 0,
                       spec = 0, sens = 0, pos_pred = 0, neg_pred = 0,
                       pos_class = "", conf = "")

for(i in c(1:32)){
  # get current team in data
  team <- svm_data$team[i]

  # get the run pass values for prediciton out of the model
  svm_data$pass_1[i] <- svm_model_info$conf_mat[[i]]$table[1,1]
  svm_data$pass_0[i] <- svm_model_info$conf_mat[[i]]$table[2,1]
  svm_data$run_1[i] <- svm_model_info$conf_mat[[i]]$table[2,2]
  svm_data$run_0[i] <- svm_model_info$conf_mat[[i]]$table[1,2]

  # get mcneymar p-value
  svm_data$mn_pvalue[i] <- svm_model_info$conf_mat[[i]]$overall[7]

  # get accuracy p-value
  svm_data$acc_pvalue[i] <- svm_model_info$conf_mat[[i]]$overall[6]

  # get sensitivity, specificity, possitive predicted value and negative predicted value
  svm_data$sens[i] <- svm_model_info$conf_mat[[i]]$byClass[1]
  svm_data$spec[i] <- svm_model_info$conf_mat[[i]]$byClass[2]
  svm_data$pos_pred[i] <- svm_model_info$conf_mat[[i]]$byClass[3]
  svm_data$neg_pred[i] <- svm_model_info$conf_mat[[i]]$byClass[4]

  # get which factor level was the "positive" class
  svm_data$pos_class[i] <- svm_model_info$conf_mat[[i]]$positive

  # get the bye week for each team
  svm_data$bye_week[i] <- svm_model_info$bye_week[[i]]

  # store the model S4 object in a temp variable for access
  temp <- svm_model_info$model[[i]] %>% extract_fit_parsnip() %>% .$fit
```

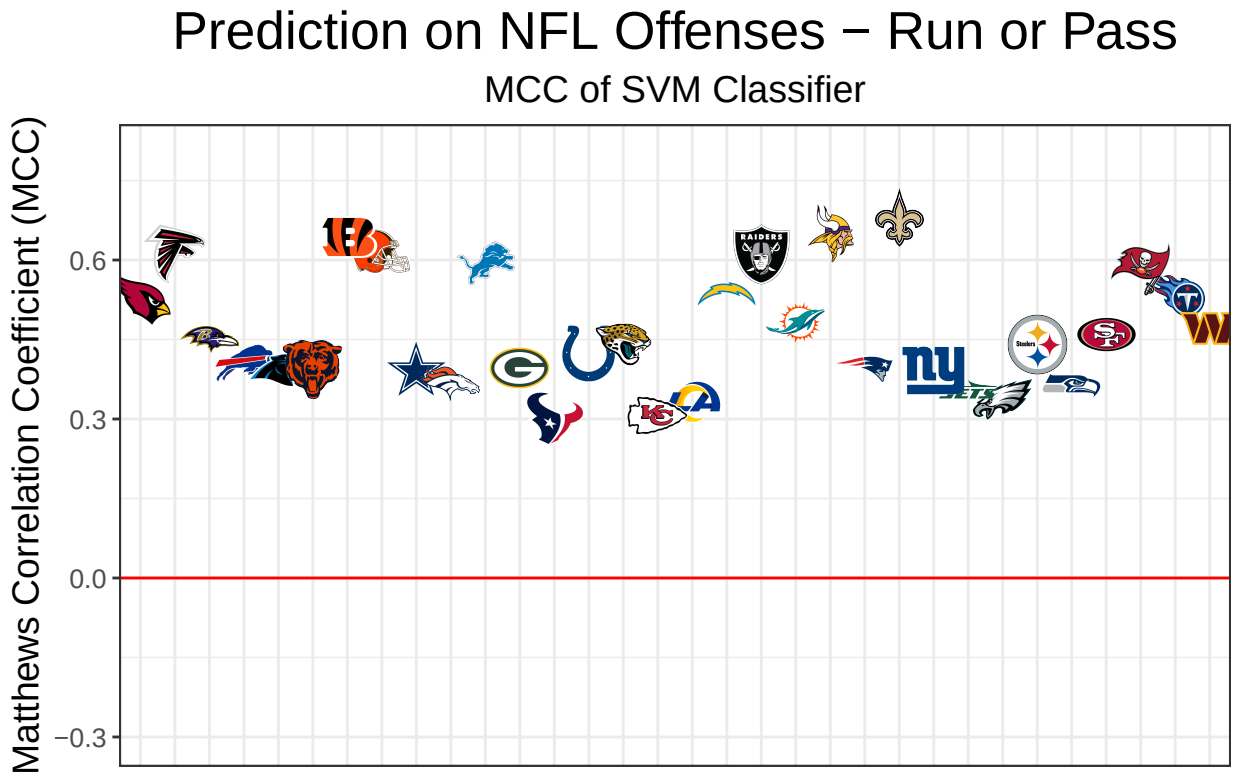
```r
  # get model training error
  svm_data$train_error[i] <- temp@error

  # get number of vectors used in model
  svm_data$num_vec[i] <- temp@nSV

  # get the conference for each team
  svm_data$conf[i] <- ifelse(team %in% afc, "AFC", "NFC")
}

# finalize df with win loss joined on, calculate MCC
svm_data <- svm_data %>% left_join(run_pass_prop, by = "team") %>%
  left_join(win_loss, by = "team") %>%
  mutate(play_prop = ifelse(tot_run > tot_pass, "More Runs", "More Passes"),
         mcc = ((run_1 * pass_1) - (run_0 * pass_0))/ sqrt((pass_1 + pass_0) * (pass_1 + run_0) * (run_
         wining_record = ifelse(wins > losses, "Winning Team", "Losing Team"))
```

Positive class = "pass" Negative class = "run" Sensitivity refers to pass Specificity refers to run

```r
# this number will scale the image sizes proportionally
s <- 1.25
spec_sens_svm <- svm_data %>%
  ggplot(aes(y = sens, x = spec)) +
  # draw the background lines. sequence for intercept will need changing
  ggplot2::geom_abline(slope = -1, intercept = seq(2, 0.6, -0.1), alpha = .2) +
  # draw the red dashed mean lines for the data
  nflplotR::geom_mean_lines(aes(x0 = spec , y0 = sens)) +
  geom_nfl_logos(aes(team_abbr = team, width = 0.075 * s, height = 0.1 * s)) +
  labs(title = "Prediction on NFL Offenses - Run or Pass",
       subtitle = "Sensitivity and Specificity of SVM Classifier",
       y = "Pass Prediction % (Sensitivity)",
       x = "Run Prediction % (Specificity)",
       caption = "data from NFL Big Data Bowl: 2022 regular season weeks1-9") +
  theme_bw() +
  theme(
    # this just makes the background flat white
    plot.background = element_rect(fill = "white"),
    panel.grid = element_blank(),
    plot.title = element_text(hjust = 0.5, size = 20),
    plot.subtitle = element_text(hjust = 0.5, size = 14),
    axis.text = element_text(size = 10),
    axis.title = element_text(size = 14)
  )
spec_sens_svm
```




```r
s <- 1
mcc_svm <- svm_data %>%
  ggplot(aes(y = mcc, x = team)) +
  geom_nfl_logos(aes(team_abbr = team, width = 0.075 * s, height = 0.1 * s)) +
  geom_hline(yintercept = 0, color = "red") +
  labs(title = "Prediction on NFL Offenses - Run or Pass",
       subtitle = "MCC of SVM Classifier",
       y = "Matthews Correlation Coefficient (MCC)",
       x = "",
       caption = "data from NFL Big Data Bowl: 2022 regular season weeks1-9") +
  scale_y_continuous(limits = c(-0.3, 0.8)) +
  theme_bw() +
  theme(axis.ticks.x = element_blank(),
        axis.text.x = element_blank(),
        plot.title = element_text(hjust = 0.5, size = 20),
        plot.subtitle = element_text(hjust = 0.5, size = 14),
        axis.text.y = element_text(size = 10),
        axis.title.y = element_text(size = 14))
mcc_svm
```




```
#Create a data frame with indicators for pass and run as well as
#the Mcnemar's Test P-Value, accuracy p-value, sensitivity, specificity,etc
log_data <- data.frame(team=unique(log_model_info$nfl_team),
                       pass_1= -1, pass_0= -1, run_1=-1, run_0=-1,
                       mn_pvalue=1, acc_pvalue=1, spec=0, sens=0,
                       pos_pred=0, neg_pred=0, pos_class="")

#Calculate the values that belong in the new columns

for(i in c(1:32)){

  #Extract the current team
  team <- log_data$team[i]

  log_data$pass_1[i] <- log_model_info$conf_mat[[match(team,log_model_info$nfl_team)]]$table[1,1]
  log_data$pass_0[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$table[2,1]
  log_data$run_1[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$table[2,2]
  log_data$run_0[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$table[1,2]
  log_data$mn_pvalue[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$overall[7]
  log_data$acc_pvalue[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$overall[6]
  log_data$sens[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[1]
  log_data$spec[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[2]
  log_data$pos_pred[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[3]
  log_data$neg_pred[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$byClass[4]
  log_data$pos_class[i] <- log_model_info$conf_mat[[match(team, log_model_info$nfl_team)]]$positive
}
```

```
log_data <- log_data %>%
  mutate(log_mcc = ((run_1 * pass_1) - (run_0 * pass_0))/ sqrt((pass_1 + pass_0) * (pass_1 + run_0) * (
```

For the Tables the only way to save them was as an HTML file. This was then opened up in a web browser to view and screen grabbed to get the images used in the paper and powerpoint slides.

```
afc_table <- svm_data %>% rename(svm_mcc = 'mcc') %>% filter(conf == "AFC") %>%
  select(team, wins, svm_mcc) %>%
  left_join(log_data %>% select(team, log_mcc),
            by = "team") %>%
  left_join(rf_data, by = "team") %>%
  select(team, win_p, log_mcc, svm_mcc, full_MCC) %>%
  mutate(win_p = round(win_p * 100, 2)) %>%
  arrange(desc(win_p))

nfc_table <- svm_data %>% rename(svm_mcc = 'mcc') %>% filter(conf == "NFC") %>%
  select(team, wins, svm_mcc) %>%
  left_join(log_data %>% select(team, log_mcc),
            by = "team") %>%
  left_join(rf_data, by = "team") %>%
  select(team, win_p, log_mcc, svm_mcc, full_MCC) %>%
  mutate(win_p = round(win_p * 100, 2)) %>%
  arrange(desc(win_p))

nfc_table <- nfc_table %>% rowwise() %>%
  mutate(best = max(c(log_mcc, svm_mcc, full_MCC)),
         worst = min(c(log_mcc, svm_mcc, full_MCC)))

nfc_comp <- nfc_table %>% rename("Team" = team,
                      "Win %" = win_p,
                      "Logistic MCC" = log_mcc,
                      "SVM MCC" = svm_mcc,
                      "RF MCC" = full_MCC) %>%
  select(-best, -worst) %>%
  kbl(align = rep('c', 16), format = "latex", booktabs = T) %>%
  kable_styling(font_size = 15, full_width = F) %>%
  column_spec(1, image = spec_image(teams_colors_logos$team_logo_espn[match(nfc_table$team, teams_colors
  column_spec(2, bold = T) %>%
  column_spec(3, bold = T, background = case_when(nfc_table$log_mcc == nfc_table$best ~ "lightgreen",
                                                  nfc_table$log_mcc == nfc_table$worst ~ "#ff8282",
                                                  T ~ "white")) %>%
  column_spec(4, bold = T, background = case_when(nfc_table$svm_mcc == nfc_table$best ~ "lightgreen",
                                                  nfc_table$svm_mcc == nfc_table$worst ~ "#ff8282",
                                                  T ~ "white")) %>%
  column_spec(5, bold = T, background = case_when(nfc_table$full_MCC == nfc_table$best ~ "lightgreen",
                                                  nfc_table$full_MCC == nfc_table$worst ~ "#ff8282",
                                                  T ~ "white"))


afc_table <- afc_table %>% rowwise() %>%
  mutate(best = max(c(log_mcc, svm_mcc, full_MCC)),
         worst = min(c(log_mcc, svm_mcc, full_MCC)))
```

```r
afc_comp <- afc_table %>% rename("Team" = team,
                      "Win %" = win_p,
                      "Logistic MCC" = log_mcc,
                      "SVM MCC" = svm_mcc,
                      "RF MCC" = full_MCC) %>%
  select(-best, -worst) %>%
  kbl(align = rep('c', 16)) %>%
  kable_styling(font_size = 15, full_width = F) %>%
  column_spec(1, image = spec_image(teams_colors_logos$team_logo_espn[match(afc_table$team, teams_colors
  column_spec(2, bold = T) %>%
  column_spec(3, bold = T, background = case_when(afc_table$log_mcc == afc_table$best ~ "lightgreen",
                                                  afc_table$log_mcc == afc_table$worst ~ "#ff8282",
                                                  T ~ "white")) %>%
  column_spec(4, bold = T, background = case_when(afc_table$svm_mcc == afc_table$best ~ "lightgreen",
                                                  afc_table$svm_mcc == afc_table$worst ~ "#ff8282",
                                                  T ~ "white")) %>%
  column_spec(5, bold = T, background = case_when(afc_table$full_MCC == afc_table$best ~ "lightgreen",
                                                  afc_table$full_MCC == afc_table$worst ~ "#ff8282",
                                                  T ~ "white"))
```

```r
nfc_comp %>% save_kable(file = "NFC_Table.html", self_contained = T)

afc_comp %>% save_kable(file = "AFC_Table.html", self_contained = T)
```